Software Requirements Specification

for

Smart Citizen Service Portal

Prepared by:

Vansh Bansal — 202401100200336
Utkarsh Mishra — 202401100200328
Utkarsh Pratap Singh — 202401100200330
Vaibhav Singh — 202401100200334

KIET Deemed to be University

Table of Contents

1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 1
2.1 Product Perspective 1
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 2
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 3
4.1 Authentication 3
4.2 Citizen Services 4
4.3 Applications 4
4.4 Complaints 5
4.5 Notifications 5
4.6 Officer Management 5
4.7 Administration 5
4.8 AI Features 6
5. Other Nonfunctional Requirements 6
5.1 Performance Requirements 6
5.2 Safety Requirements 6
5.3 Security Requirements 6
5.4 Software Quality Attributes 7
5.5 Business Rules 7
6. Other Requirements 7
6.1 Database Requirements 7
6.2 API Requirements 8
6.3 AI/ML Requirements 8
6.4 Future Enhancements 8
Appendix A: Glossary 9
Appendix B: Analysis Models 9
Appendix C: To Be Determined List 11

Revision History

Name	Date	Reason For Changes	Version
Team	Aug 2026	Initial SRS draft (concise project-overview format)	1.0
Team	Aug 2026	Restructured into IEEE 830 template with priority/stability tags and traceability	2.0
Vansh Bansal (202401100200336) et al.	10th September, 2026	Reformatted into official KIET IEEE-830 template	3.0

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for Smart Citizen Service Portal, Version 1.0. It covers the complete web-based platform: the citizen-facing portal, the officer interface, the administrator dashboard, and the supporting AI service. It does not cover any future mobile application or government-system integration, both of which are listed as future enhancements in Section 6.

1.2 Document Conventions

Headings are numbered hierarchically (e.g., 4.1, 4.1.1) following this template. Every functional requirement carries a unique, meaningful identifier (e.g., FR-AUTH-01) rather than a bare sequence number, so that a requirement can be referenced unambiguously from test cases or design documents. The keyword “shall” denotes a mandatory requirement; “should” denotes a recommended but non-binding one. Requirements at the feature level (Section 4) inherit the priority stated for that feature unless individually noted otherwise.

1.3 Intended Audience and Reading Suggestions

This document is intended for the development team, the project guide/evaluator, and any future maintainers of the system. Developers should read Sections 3 through 6 in full. Evaluators may read Sections 1 and 2 for context and Section 4 for requirement completeness. Testers should use Section 4 and Appendix B to derive test cases. Readers are encouraged to begin with Section 1 and Section 2 before moving into the detailed requirements in Sections 3-6.

1.4 Product Scope

Smart Citizen Service Portal is a proposed AI-assisted e-governance web platform that lets citizens browse civic services, submit applications and complaints, and track their resolution, while giving government officers and administrators the tools to manage and resolve these requests efficiently. The AI component classifies and prioritizes complaints, detects duplicates, and powers a citizen-facing chatbot restricted to an approved knowledge base. This is an academic proposal built for demonstration purposes; it is not integrated with any real government department or live government database.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018, Systems and Software Engineering — Life Cycle Processes — Requirements Engineering.
- React documentation — https://react.dev
- FastAPI documentation — https://fastapi.tiangolo.com
- PostgreSQL documentation — https://www.postgresql.org/docs
- Team project proposal, "Smart Citizen Service Portal" (internal document, KIET Deemed to be University, 2026).

2. Overall Description

2.1 Product Perspective

The portal is a new, standalone, three-tier web system: a React-based frontend, a FastAPI backend exposing REST APIs, and a PostgreSQL database, supported by a Python-based AI service for classification, prediction,

and conversational assistance. It is not a follow-on to any existing product and has no dependency on legacy government infrastructure.

2.2 Product Functions

- Citizen registration, authentication, and profile management
- Browsing and applying for civic services with document upload
- Complaint submission, categorization, and status tracking
- Officer workflows for reviewing and resolving assigned items
- Administrative management of services, departments, users, and analytics
- AI-assisted classification, prioritization, duplicate detection, and citizen chatbot support

2.3 User Classes and Characteristics

User Class	Characteristics / Responsibilities
Citizen	General public, varying technical literacy; uses services, applications, complaints, tracking
Officer	Departmental staff; reviews and resolves tasks assigned to them
Admin	Platform administrator; manages system, services, departments, and users

2.4 Operating Environment

Client: any modern evergreen web browser (Chrome, Firefox, Edge, Safari) on desktop or mobile, with an active internet connection. Server: a Linux-based host running Python 3.11+ (FastAPI) and PostgreSQL 14+. No native mobile app or special client hardware is required for this release.

2.5 Design and Implementation Constraints

Layer	Technology
Frontend	React.js, JavaScript, Tailwind CSS, Axios, React Router
Backend	Python, FastAPI, REST APIs, JWT authentication, SQLAlchemy
Database	PostgreSQL
AI/ML	Scikit-learn, NLP (TF-IDF/cosine similarity, Sentence Transformers), LLM API + RAG for chatbot

- The team is limited to 4 student developers and an academic-term timeline, constraining scope to the MVP feature set.
- Technologies are fixed to the above stack; a given technology is used only where relevant, not forced into every module.

2.6 User Documentation

An in-app help/FAQ section and contextual tooltips will be provided for citizen users. A short administrator guide (PDF) will document service/department configuration for admins. A README will document setup and API usage for developers maintaining the project after handover.

2.7 Assumptions and Dependencies

- Users require an internet connection to access the portal.
- Integration with real government systems and databases is outside the MVP scope.

- AI predictions assist officers and administrators but do not make final administrative decisions.
- Only authenticated and authorized users can access protected citizen or administrative data.
- Availability of a third-party LLM API is assumed for the chatbot feature.

3. External Interface Requirements

3.1 User Interfaces

The portal provides three responsive, browser-based interfaces — Citizen, Officer, and Admin — sharing a common React component library and Tailwind CSS design system. Screens include registration/login, service catalog, application/complaint forms with file upload, status-tracking views, an AI chat widget, and role-specific dashboards. Interfaces target usability by non-technical citizen users; details of screen-level design will be documented in a separate UI specification.

3.2 Hardware Interfaces

None. The system is a purely software web application; no dedicated hardware is required beyond standard client devices (desktop/mobile) and commodity server infrastructure.

3.3 Software Interfaces

Interface	Description
React Frontend ↔ FastAPI backend	JSON over REST, authenticated via JWT bearer tokens
FastAPI backend ↔ PostgreSQL	SQLAlchemy ORM over a standard database connection
Backend ↔ AI Service	Internal REST/function calls for classification, priority, and duplicate detection
AI Service ↔ LLM API	HTTPS calls to an external LLM provider for chatbot responses (RAG over an approved knowledge base)

3.4 Communications Interfaces

All client-server communication uses HTTPS. In-app notifications are delivered via the portal's notification module; email notifications (e.g., password-reset OTP) use a standard SMTP or transactional-email provider. No other communications protocols are required for this release.

4. System Features

4.1 Authentication

4.1.1 Description and Priority

Registration, login, and password-recovery flows with role-based access for Citizen, Officer, and Admin. Priority: High.

4.1.2 Stimulus/Response Sequences

- User submits registration details → system validates and creates the account.
- User submits login credentials → system validates and issues a JWT session token.
- User requests a password reset → system sends a verification OTP/link by email.

4.1.3 Functional Requirements

FR-AUTH-01: The system shall allow new citizens to register using name, email/phone, and password.

FR-AUTH-02: The system shall authenticate users via email/username and password and issue a JWT on success.

FR-AUTH-03: The system shall enforce distinct login flows and permissions for Citizen, Officer, and Admin roles.

FR-AUTH-04: The system shall allow users to reset a forgotten password via a verified email/OTP flow.

4.2 Citizen Services

4.2.1 Description and Priority

Browsing the service catalog and managing a citizen profile. Priority: High.

4.2.2 Stimulus/Response Sequences

- Citizen searches the service catalog → system returns matching services.
- Citizen edits profile fields → system validates and saves the update.

4.2.3 Functional Requirements

FR-CIT-01: The system shall allow citizens to view and search the catalog of available civic services.

FR-CIT-02: The system shall allow citizens to view and edit their profile information.

FR-CIT-03: The system shall support adding new service types without requiring changes to core application logic.

4.3 Applications

4.3.1 Description and Priority

Submitting and tracking service applications. Priority: High.

4.3.2 Stimulus/Response Sequences

- Citizen submits an application with documents → system creates the record and issues an application ID.
- Officer/admin reviews an application → system records approval, rejection, or change request.

4.3.3 Functional Requirements

FR-APP-01: The system shall allow authenticated citizens to apply for a service by submitting required details and documents.

FR-APP-02: The system shall assign a unique application ID upon successful submission.

FR-APP-03: The system shall allow citizens to track the current status of their submitted applications.

FR-APP-04: The system shall allow officers/admins to approve, reject, or request changes to an application.

4.4 Complaints

4.4.1 Description and Priority

Complaint submission, lifecycle management, and resolution. Priority: High.

4.4.2 Stimulus/Response Sequences

- Citizen submits a complaint → system creates the record and queues it for AI analysis.
- Officer updates complaint status → system logs the change and notifies the citizen.

4.4.3 Functional Requirements

FR-COMP-01: The system shall allow authenticated citizens to submit a complaint containing a category, description, location, and optional supporting image.

FR-COMP-02: The system shall allow authorized officers to update the status of complaints assigned to them.

FR-COMP-03: The system shall support a complaint lifecycle of Submitted, Under Review, Assigned, In Progress, Resolved, and Closed.

FR-COMP-04: The system shall allow officers/admins to add remarks to a complaint at each status change.

FR-COMP-05: The system shall allow citizens to view the full status history of their complaint.

4.5 Notifications

4.5.1 Description and Priority

Keeping citizens informed of status changes. Priority: Medium.

4.5.2 Stimulus/Response Sequences

- Application/complaint status changes → system generates a notification for the citizen.

4.5.3 Functional Requirements

FR-NOT-01: The system shall notify citizens when the status of their application or complaint changes.

FR-NOT-02: The system shall allow citizens to view a list of their past notifications within the portal.

4.6 Officer Management

4.6.1 Description and Priority

Tools for officers to act on assigned work. Priority: High.

4.6.2 Stimulus/Response Sequences

- Officer opens their task list → system displays assigned complaints/applications.
- Officer marks a task resolved → system updates status and notifies the citizen.

4.6.3 Functional Requirements

FR-OFF-01: The system shall allow officers to view a list of complaints/applications assigned to them.

FR-OFF-02: The system shall allow officers to upload supporting documents while resolving a task.

FR-OFF-03: The system shall allow officers to mark an assigned complaint or application as resolved.

4.7 Administration

4.7.1 Description and Priority

Platform configuration and oversight for administrators. Priority: High.

4.7.2 Stimulus/Response Sequences

- Admin creates/updates a service or department → system persists the change and reflects it in the citizen catalog.
- Admin reassigns a complaint → system updates ownership and notifies the new officer.

4.7.3 Functional Requirements

FR-ADMIN-01: The system shall allow administrators to create, update, and deactivate services and departments.

FR-ADMIN-02: The system shall allow administrators to manage citizen and officer accounts.

FR-ADMIN-03: The system shall allow administrators to assign or reassign complaints to a department or officer.

FR-ADMIN-04: The system shall provide administrators with a dashboard summarizing applications and complaints.

FR-ADMIN-05: The system shall allow administrators to generate basic reports on service and complaint performance.

4.8 AI Features

4.8.1 Description and Priority

AI-assisted classification, prediction, and citizen assistance. Priority: High (core differentiator).

4.8.2 Stimulus/Response Sequences

- Citizen submits a complaint → AI service returns predicted category, department, and priority.
- Citizen asks the chatbot a question → chatbot answers using the approved knowledge base (and status data if authenticated).

4.8.3 Functional Requirements

FR-AI-01: The system shall analyze complaint descriptions and predict a complaint category and relevant department.

FR-AI-02: The system shall predict a priority level (Low, Medium, High, Critical) for each submitted complaint.

FR-AI-03: The system shall identify potentially duplicate or related complaints using semantic similarity.

FR-AI-04: The system shall provide an AI chatbot that answers citizen questions about services, required documents, and application procedures using an approved knowledge base.

FR-AI-05: The system shall allow the AI chatbot to answer application/complaint status queries only for the authenticated requesting citizen.

FR-AI-06: The system shall present AI-predicted category and priority to officers as a suggestion that can be overridden.

Note: This feature list targets 30–40 high-value, testable requirements rather than exhaustive enumeration, consistent with the project's academic scope.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

NFR-PERF-01: The system should provide normal API responses within an acceptable response time under expected project load (proposed target, not contractually guaranteed).

5.2 Safety Requirements

Not applicable. The system does not control physical equipment and poses no direct physical-safety risk; “safety” here is limited to data integrity and correct handling of citizen-submitted information.

5.3 Security Requirements

NFR-SEC-01: The system shall enforce role-based access control for Citizen, Officer, and Administrator accounts.

NFR-SEC-02: The system shall store passwords using a secure hashing algorithm and never store them in plain text.

NFR-PRIV-01: The system shall restrict access to personal citizen data to the citizen themselves and authorized officers/admins.

- HTTPS for all client-server communication
- Server-side input validation on all API endpoints
- Secure handling and storage of uploaded files
- Authorization checks on every protected API route
- Protection against common web vulnerabilities (e.g., SQL injection, XSS, CSRF)
- Audit logging of key administrative and status-change actions
- Safeguards against prompt-injection attempts on the AI chatbot

5.4 Software Quality Attributes

NFR-AVAIL-01: The system should remain available during normal demonstration and evaluation periods, with no formal SLA.

NFR-SCAL-01: The architecture should allow additional services and departments to be added without major structural change.

NFR-USE-01: The citizen-facing interface shall be usable by non-technical users with minimal guidance.

NFR-MAINT-01: The codebase shall be modular, separating frontend, backend, and AI service concerns for ease of maintenance.

5.5 Business Rules

- AI-generated category, priority, and duplicate flags are advisory; only an officer/admin action changes official record status.
- A complaint or application may only be updated by an officer to whom it is assigned, or by an administrator.
- The AI chatbot must answer only from the approved knowledge base and must not invent policies or procedures.

6. Other Requirements

6.1 Database Requirements

Entity	Purpose
User	Authentication and role information
Citizen / Officer / Admin	Role-specific profile details linked to User
Department	Organizational unit handling services and complaints
Service	Available civic services
Application	Citizen service requests
Complaint	Citizen grievances
ComplaintUpdate	Status/remark history for a complaint
Document	Uploaded files linked to applications/complaints

Entity	Purpose
Notification	Status-change alerts sent to citizens
Feedback	Citizen feedback on resolved services
AIPrediction	AI-generated classification/priority information

6.2 API Requirements

The following are proposed REST API endpoints and may evolve during implementation.

Module	Method	Endpoint	Purpose
Auth	POST	/auth/register	Register user
Auth	POST	/auth/login	Login
Services	GET	/services	List services
Applications	POST	/applications	Submit application
Applications	GET	/applications/{id}	Track application
Complaints	POST	/complaints	Submit complaint
Complaints	GET	/complaints/{id}	View complaint
Complaints	PUT	/complaints/{id}/status	Update status
AI	POST	/ai/classify-complaint	Classify complaint
AI	POST	/ai/detect-duplicate	Detect duplicate
AI	POST	/ai/chat	Chatbot
Admin	GET	/admin/analytics	Dashboard analytics

6.3 AI/ML Requirements

AI Feature	Input	Output	Suggested Approach
Classification	Complaint text	Category/Department	NLP classifier
Priority	Complaint details	Priority level	ML classification
Duplicate Detection	Complaint text	Similarity score	Embeddings / cosine similarity
Chatbot	User query	Response	LLM + RAG over knowledge base

Evaluation metrics: Classification — Accuracy, Precision, Recall, F1-score. Duplicate detection — Precision/Recall. Chatbot — response relevance and factual consistency against the knowledge base.

AI recommendations assist authorities but do not replace human decision-making.

6.4 Future Enhancements (Out of MVP Scope)

- Mobile application
- Multilingual support
- Voice assistant
- GIS/map integration
- IoT-based civic monitoring

- Advanced predictive analytics
- Government-system integrations
- Automated document verification (OCR)

Appendix A. Glossary

Term	Meaning
SRS	Software Requirements Specification
MVP	Minimum Viable Product
JWT	JSON Web Token
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control

Appendix B. Analysis Models

B.1 System Architecture

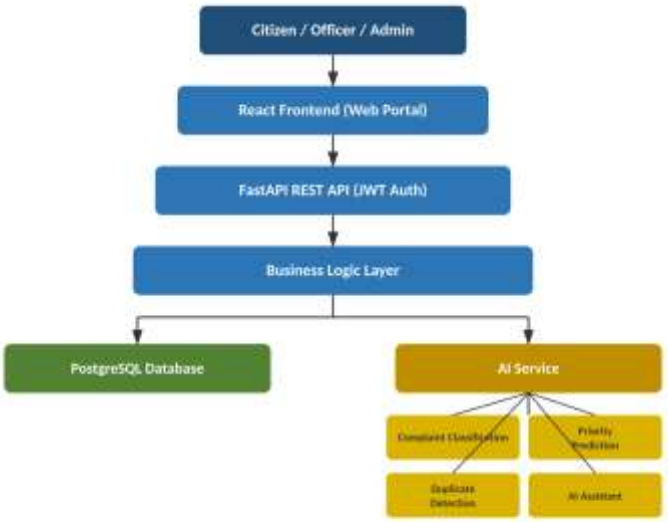


Figure 1: Proposed layered architecture

Layer	Responsibility
React Frontend	Citizen, officer, and admin UI; calls backend REST APIs
FastAPI REST API	Request handling, authentication (JWT), routing
Business Logic Layer	Core application rules, workflow, validation
PostgreSQL	Persistent storage of all application data
AI Service	Classification, priority prediction, duplicate detection, chatbot

B.2 Entity-Relationship Diagram

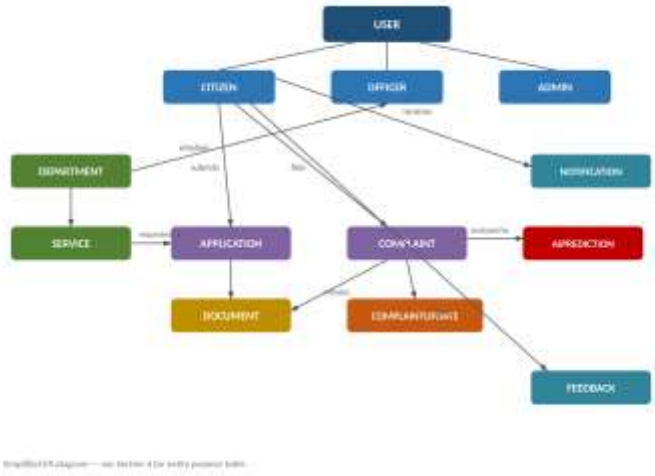


Figure 2: Simplified entity-relationship diagram

B.3 Key Use Cases / Workflows

ID / Actor	Precondition	Main Flow	Postcondition
UC-01 Citizen Registration/Login	Valid email/phone	Register or log in → validate credentials → issue JWT	Citizen has an authenticated session
UC-02 Citizen Apply for a Service	Citizen authenticated	Select service → fill details, upload docs → submit	Application created with unique ID and tracked status
UC-03 Citizen Submit Complaint	Citizen authenticated	Enter category, description, location, image → submit	Complaint created and queued for AI analysis
UC-04 System AI Classification	New complaint submitted	AI analyzes description → predicts category, dept., priority	Complaint tagged with AI suggestions
UC-05 Officer Resolve Complaint	Complaint assigned to officer	Review → update status/remarks → mark resolved	Status updated; citizen notified
UC-06 Citizen Track Request	Existing application/complaint	Open tracking view → view status and history	Citizen sees latest status
UC-07 Citizen AI Assistant	Question about a service/request	Ask chatbot → chatbot retrieves answer from knowledge base (+status if authenticated)	Citizen gets a policy-consistent answer
UC-08 Admin Management	Admin authenticated	Manage services/departments/users → review analytics → reassign complaints	Configuration and assignments updated

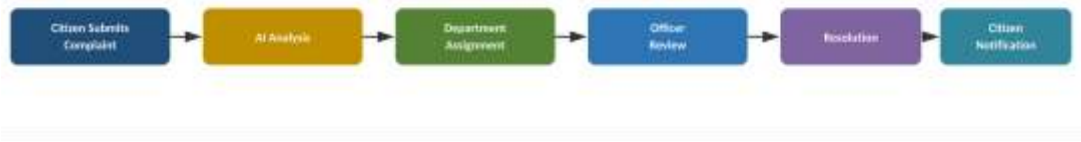


Figure 3: Combined complaint-resolution workflow

Appendix C. To Be Determined List

- TBD-1: Final choice of LLM API provider for the chatbot (cost/latency trade-off not yet finalized).
- TBD-2: Exact OTP/email provider for password-reset flow.
- TBD-3: Concrete performance targets for NFR-PERF-01, pending load-testing during development.
- TBD-4: Final confirmation of hosting environment (college lab server vs. free-tier cloud host).